Reviewer Pack — Continued-Fraction Spike of Truth-Table Rational Caps DNF_mi…

Entry 6ead43796693 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 16:32:37 UTC

Continued-Fraction Spike of Truth-Table Rational Caps DNF_min

Entry ID: `6ead43796693` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-28 16:32:37 UTC

1. Conjecture

Field A (mathematical branch): Diophantine approximation via partial-quotient extremes: the Gauss-Kuzmin-Lévy distribution of continued-fraction coefficients and the rare-event tail of super-polylog partial quotients (Khinchin 1936; Bosma-Jager-Wiedijk; Hensley's bounds). A classical analytic-number-theory invariant computed by one Euclidean-algorithm pass; distinct from Mahler measure / Lehmer heights and from spectral approaches, and rarely deployed in meta-complexity. **Field B** (complexity object): DNF_min(f) for f:{0,1}^n→{0,1} given by truth table — the Hirahara–Oliveira–Santhanam restricted-MCSP proxy in average-case-to-worst-case meta-complexity reductions, and its behavior under random p-restrictions (the Impagliazzo-Wigderson avg-to-worst hardness object).

Statement:

> For f:{0,1}^n→{0,1} let TT(f)∈{0,…,2^N−1} with N=2^n be its lex-ordered truth-table integer, α(f):=TT(f)/(2^N+1)∈[0,1)∩ℚ, and S(f):=max_{i≥1} a_i over the Euclidean continued-fraction expansion [a_0;a_1,…,a_L] of α(f). CONJECTURE: there exist absolute constants c_1,c_2>0 such that for every n≥4 and every f, if S(f) ≥ n^{c_1·log n} then DNF_min(f) ≤ 2^n·n / S(f)^{c_2}. Equivalently, a super-polylog Diophantine spike in α(f) is a one-sided structural certificate of polynomially-bounded DNF complexity, while the spike event has Gauss-Kuzmin probability < 1/poly(N) on random TTs.

Rationale (proposer's reasoning):

> A coefficient a_i ≥ M in the CF of α(f)=p/q means p/q is exceptionally close to a rational p'/q' of denominator q' ≈ N/M, which forces the congruence TT(f)·q' ≈ k(2^N+1) and makes TT(f) approximately periodic with period O(N/q'); approximate periodicity at period O(log q') in N=2^n bits forces f to approximately depend on a junta of O(log q') coordinates and hence to admit a small DNF. This realizes the Impagliazzo-Wigderson avg-to-worst paradigm structurally: outside the rare Gauss-Kuzmin spike tail, no Diophantine shortcut to low-DNF can occur, so DNF-hardness on the typical regime is the worst-case object. Because the implication is one-sided 'rare signature ⇒ low complexity' (not 'large signature ⇒ high complexity'), the property is not large and sidesteps Razborov-Rudich naturalness.

Taxonomy category: `AVG_TO_WORST_CASE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `875c07296398840f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across n∈{4,5,6,7}, every function f with S(f)≥n^{log n} encountered in (i) the ≤4-term/≤4-literal DNF enumeration, (ii) 5000 random TTs per n, and (iii) the adversarial m∈{3,5,7,11,13} sweep must satisfy DNF_min(f) ≤ $2^{n \cdot n/S(f)}${0.1}; aggregate support fraction over all spike-events ≥ 0.99.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `continued fraction partial quotients truth table circuit complexity` - `DNF minimization Diophantine approximation meta-complexity MCSP` - `Khinchin partial quotients Boolean function complexity restriction`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1908.00121v4] Continued fractions over non-Euclidean imaginary quadratic rings - [http://arxiv.org/abs/2209.08318v3] Hausdorff dimension of sets with restricted, slowly growing partial quotients in semi-regular continued fractions - [http://arxiv.org/abs/1609.06986v1] More on Diophantine sextuples - [http://arxiv.org/abs/1603.03800v3] Diophantine approximation on matrices and Lie groups - [http://arxiv.org/abs/2412.05161v1] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/2007.10924v3] Partial Boolean functions with exact quantum 1-query complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

```
def continued_fraction(numerator, denominator):
    if denominator == 0:
        raise ValueError("Denominator cannot be zero")
    a0 = numerator // denominator
    remainder = numerator % denominator
    cf = [a0]
    while remainder != 0:
        numerator = denominator
        denominator = remainder
        a = numerator // denominator
        remainder = numerator % denominator
        cf.append(a)
    return cf

def euclidean_algorithm(numerator, denominator):
    if gcd(numerator, denominator) == 1:
        return continued_fraction(numerator, denominator)

def decode_tt(tt, n):
    f = [0] * (2**n)
    for i in range(2**n):
        f[i] = tt & 1
        tt >>= 1
    return f

def quine_mccluskey(f):
    # Simplify the DNF using Quine-McCluskey algorithm
    pass  # Placeholder, actual implementation required

def dnf_min(f):
    # Compute the minimum size of a DNF representation of f
    pass  # Placeholder, actual implementation required

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 5, 6, 7]
    results = []

    for n in n_values:
        instances_tested = 0
        support_fraction = 0

        # Enumerate all DNFs with ≤4 terms and ≤4 literals
        dnf_count = 10**5
        for _ in range(dnf_count):
            instances_tested += 1
            # Generate a random DNF function
            f = [random.choice([0, 1]) for _ in range(2**n)]
            tt = int("".join(map(str, f)), 2)
            alpha_f = tt / (2**(2*n) + 1)
            cf = euclidean_algorithm(int(alpha_f * (2**(2*n) + 1)), 2**(2*n) + 1)
            S_f = max(cf[1:])

            if S_f >= n ** (n.bit_length()):
                dnf_complexity = dnf_min(f)
```

```
            if dnf_complexity > 2**n * n / S_f**0.1:
                return {
                    "metric_name": "support_fraction",
                    "metric_value": support_fraction,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"n={n}, DNF_min(f) > 2^n·n/S(f)^0.1"
                }
            support_fraction += 1

    # Sample 5000 uniformly random TTs
    for _ in range(5000):
        instances_tested += 1
        tt = random.randint(0, 2**(2*n) - 1)
        alpha_f = tt / (2**(2*n) + 1)
        cf = euclidean_algorithm(int(alpha_f * (2**(2*n) + 1)), 2**(2*n) + 1)
        S_f = max(cf[1:])

        if S_f >= n ** (n.bit_length()):
            return {
                "metric_name": "support_fraction",
                "metric_value": support_fraction,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": f"n={n}, DNF_min(f) > 2^n·n/S(f)^0.1"
            }

    # Adversarial sweep
    for m in [3, 5, 7, 11, 13]:
        for k in range(1, 2**(2*n), m):
            if gcd(k, m) == 1:
                instances_tested += 1
                tt = round(k * (2**(2*n) + 1) / m)
                f = decode_tt(tt, n)
                alpha_f = tt / (2**(2*n) + 1)
                cf = euclidean_algorithm(int(alpha_f * (2**(2*n) + 1)), 2**(2*n) + 1)
                S_f = max(cf[1:])

                if S_f >= n ** (n.bit_length()):
                    dnf_complexity = dnf_min(f)
                    if dnf_complexity > 2**n * n / S_f**0.1:
                        return {
                            "metric_name": "support_fraction",
                            "metric_value": support_fraction,
                            "instances_tested": instances_tested,
                            "conjecture_holds": False,
                            "counterexample": f"n={n}, DNF_min(f) > 2^n·n/S(f)^0.1"
                        }
                    support_fraction += 1

mean_value = sum(x["metric_value"] for x in results) / len(results)
std_value = (sum((x["metric_value"] - mean_value)**2 for x in results) / len(results))**0.5
support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

return {
    "metric_name": "support_fraction",
```

```
        "metric_value": support_fraction,
        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) if sys.argv[1:] else [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(x["metric_value"] for x in results) / len(results)
    std_value = (sum((x["metric_value"] - mean_value)**2 for x in results) / len(results))**0.5
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if support_fraction >= 0.99:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={results[first_failing_seed]['metric_name']},
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_338fbe98.py", line 145, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_338fbe98.py", line 78, in run_trial
    if dnf_complexity > 2**n * n / S_f**0.1:
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: '>' not supported between instances of 'NoneType' and 'float'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError (DNF_min returned None and was compared to a float), so no data was produced and the pre-registered support condition cannot be evaluated.

Per rule 2, a crashed test yields INCONCLUSIVE. | next: Fix the DNF_min routine to always return an integer (or guard the comparison against None) and rerun the full pre-registered sweep over n∈{4,5,6,7} including the 5000 random TTs per n and adversarial m sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	296510
2	preregistration	claude_max	opus	0	0	8005
3	novelty	claude_max	opus	0	0	4605
4	novelty	claude_max	opus	0	0	10624
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20446
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16898
7	judge	claude_max	opus	0	0	5108

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 362195 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6ead43796693.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ead43796693.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6ead43796693.tar.gz` (if generated)